

```
-- Some starter code for Part 1 of CSC 330 Summer 2026 A#1
--
-- Author: Mike Zastre
--
-- The copyright for this code is held by Mike Zastre. It is
-- not to be uploaded or otherwise accessed by any online service
-- or online-facilitated tool (such as Copilot or ChatGPT, although
-- this list is not exhaustive).
```

```
text_a = "2 + 3 * 4"
text_b = "(3 + 5) * (7 - 2)"
text_c = "(4 + 6 * (2 ^ 3)) - 5 / (1 + 1)"
text_d = "(14 + 26 * (12 ^ 3)) - 55 / (10 + 11)"
```

```
data ParseNode =
    Expr ParseNode
  | Add ParseNode ParseNode
  | Sub ParseNode ParseNode

  | Term ParseNode
  | Mult ParseNode ParseNode
  | Div ParseNode ParseNode

  | Factor ParseNode
  | Power ParseNode ParseNode

  | Base ParseNode
  | Number ParseNode

  | Digit Integer
  deriving (Show, Eq)
```

```
parse_tree_a :: ParseNode
parse_tree_a =
  Add
    (Expr
      (Term
        (Factor
          (Base
            (Number
              (Digit 2))))))
      (Term
        (Mult
          (Term
            (Factor
              (Base
                (Number
                  (Digit 3))))))
          (Factor
            (Base
              (Number
                (Digit 4))))))
```

```
parse_tree_b :: ParseNode
parse_tree_b =
  Expr
    (Term
      (Mult
        (Term
          (Factor
            (Base
              (Add
```

```

(Expr
  (Term
    (Factor
      (Base
        (Number
          (Digit 3))))))
  (Term
    (Factor
      (Base
        (Number
          (Digit 5))))))
  (Factor
    (Base
      (Sub
        (Expr
          (Term
            (Factor
              (Base
                (Number
                  (Digit 7))))))
          (Term
            (Factor
              (Base
                (Number
                  (Digit 2))))))
        (Term
          (Factor
            (Base
              (Number
                (Digit 4))))))
        (Term
          (Mult
            (Term
              (Factor
                (Base
                  (Number
                    (Digit 6))))))
            (Factor
              (Base
                (Power
                  (Base
                    (Number
                      (Digit 2)))
                  (Factor
                    (Base
                      (Number
                        (Digit 3))))))))))
        (Term
          (Div
            (Term
              (Factor
                (Base
                  (Number
                    (Digit 5))))
            (Factor

```

```

(Base
  (Add
    (Expr
      (Term
        (Factor
          (Base
            (Number
              (Digit 1))))))
    (Term
      (Factor
        (Base
          (Number
            (Digit 1)))))))))

```

```

parse_tree_d :: ParseNode

```

```

parse_tree_d =

```

```

  Sub

```

```

    (Expr
      (Term
        (Factor
          (Base
            (Add
              (Expr
                (Term
                  (Factor
                    (Base
                      (Number
                        (Digit 14))))))
                (Term
                  (Mult
                    (Term
                      (Factor
                        (Base
                          (Number
                            (Digit 26))))))
                    (Factor
                      (Base
                        (Power
                          (Base
                            (Number
                              (Digit 12)))
                          (Factor
                            (Base
                              (Number
                                (Digit 3))))))))))))))
              (Term
                (Div
                  (Term
                    (Factor
                      (Base
                        (Number
                          (Digit 55))))))
                  (Factor
                    (Base
                      (Add
                        (Expr
                          (Term
                            (Factor
                              (Base
                                (Number
                                  (Digit 10))))))
                          (Term
                            (Factor

```

```
(Base
  (Number
    (Digit 11)))))))))
```

```
expr_eval :: ParseNode -> Integer
expr_eval (Digit n) = n
```

```
expr_eval (Number n) = expr_eval n
expr_eval (Base n) = expr_eval n
expr_eval (Factor n) = expr_eval n
expr_eval (Expr n) = expr_eval n
expr_eval (Term n) = expr_eval n
```

```
expr_eval (Add e1 e2) = (expr_eval e1) + (expr_eval e2)
expr_eval (Sub e1 e2) = (expr_eval e1) - (expr_eval e2)
expr_eval (Mult e1 e2) = (expr_eval e1) * (expr_eval e2)
expr_eval (Div e1 e2) = (expr_eval e1) `div` (expr_eval e2)
expr_eval (Power e1 e2) = expr_eval e1 ^ expr_eval e2
```

```
-- Code provided to students in UVic CSC 330 Summer 2026
--
-- Author: Mike Zastre
--
-- The copyright for this code is held by Mike Zastre. It is
-- not to be uploaded or otherwise accessed by any online service
-- or online-facilitated tool (such as Copilot or ChatGPT, although
-- this list is not exhaustive).
```

```
import System.IO
import System.Environment (getArgs)
import Data.List
```

```
type ProblemTuple = (Integer, [Integer])
```

```
-- Top level function: Because Haskell constrains the mixture of
-- of functions having I/O with functions having no I/O, we need
-- to use an IO monad (the "IO ()" syntax). There should be no
-- need to modify this code unless you _really_ know what you are
-- doing in Haskell. This has been written for you in order that
-- you may completely ignore I/O issues.
```

```
-- The first parameter if 'stamps' is the name of the input file;
-- the second is the name of the output file.
```

```
-- THERE SHOULD BE NO NEED TO MODIFY THE CODE UP TO THE POINT
-- WITH THE COMMENT "-- STUDENT WORK BEGINS HERE".
```

```
main :: IO ()
main = do
    args <- getArgs
    case args of
        (inFile : outFile : _) -> stamps inFile outFile
        _ -> putStrLn "Usage: <input file> <output file>"
```

```
stamps :: String -> String -> IO ()
stamps inName outName = do
    inFile <- openFile inName ReadMode
    outFile <- openFile outName WriteMode
    lineData <- hGetContents inFile
    let rawLines = lines lineData
    let problemList = parseData rawLines
    let answers = process problemList
    outputResults outFile answers
    hClose inFile
    hClose outFile
```

```
parseData :: [String] -> [ProblemTuple]
parseData [] = []
parseData (a:[])
    | a == "0" = []
    | otherwise = [(-1, [])]
parseData (a:b:as) = (width, stamps) : parseData as
    where
        width = read a :: Integer
        stamps = string2ints (words b)
```

```
string2ints :: [String] -> [Integer]
string2ints [] = []
string2ints (a:as) = (read a :: Integer) : string2ints as
```

```
ints2string :: [Integer] -> String
```

```
ints2string [] = ""
ints2string (a:[]) = show a
ints2string (a:as) = show a ++ " " ++ ints2string as

outputResults :: Handle -> [String] -> IO ()
outputResults _ [] = return ()
outputResults outFile (a:as) = do
    hPutStrLn outFile a
    outputResults outFile as

--
-- STUDENT WORK BEGINS HERE
--
-- YOU ARE FORBIDDEN TO USE EITHER "do" OR "let" IN YOUR
-- SOLUTION. THE APPEARANCE OF EITHER OF THESE KEYWORDS
-- IN YOUR CODE MAY RESULT IN A FAILING GRADE FOR THIS PART
-- OF THE ASSIGNMENT.
--

process :: [ProblemTuple] -> [String]
process [] = []
process (x:xs) = singleProblem x : process xs

singleProblem :: ProblemTuple -> String
singleProblem (maxStamps, stamps) =
    "max coverage = " ++ show(coverage maxStamps (tail stamps)) ++ " : " ++ ints2string
    (tail stamps)

coverage :: Integer -> [Integer] -> Integer
coverage maxStamps denominations = coverageFrom 1 maxStamps denominations

coverageFrom :: Integer -> Integer -> [Integer] -> Integer
coverageFrom amount maxStamps denominations
    | checkValue amount maxStamps denominations = coverageFrom (amount + 1) maxStamps d
    enominations
    | otherwise = amount - 1

-- Amount (Start 0 count up), stamps left, [stamp denominations] returns bool
checkValue :: Integer -> Integer -> [Integer] -> Bool
checkValue 0 _ _ = True -- Can always make 0
checkValue _ 0 _ = False -- Cant make anything with no stamps left
checkValue amount stampsLeft denominations =
    tryStamps amount stampsLeft denominations denominations

--Amount, stamps left, denominations, denominations again to remove first element retur
n Bool
tryStamps :: Integer -> Integer -> [Integer] -> [Integer]-> Bool
tryStamps _ _ _ [] = False
tryStamps amount stampsLeft denominations (x:xs)
    | amount - x < 0 = tryStamps amount stampsLeft denominations xs

    -- Checks stamps incrementally to find largest x that fits in the ammount
    | checkValue (amount - x) (stampsLeft - 1) denominations = True

    -- continue checking coverage with smaller amount and one less stamp
    | otherwise = tryStamps amount stampsLeft denominations xs

    -- Continue checking next x if checkValue comes back false with current x
```

```
--  
-- Here are two problem tuples to help you get started  
--
```

```
pt0 :: [ProblemTuple]  
pt0 = [(5, [2, 1, 4])]  
  
pt1 :: [ProblemTuple]  
pt1 = [(5, [4, 1, 4, 12, 21]),  
      (10, [5, 1, 7, 16, 31, 88]),  
      (6, [4, 1, 5, 7, 8])  
      ]
```